# Session 12: Structures — Notes & Code

C Language Study Notes & Code Guide

## Topic Notes: Structures in C

### What is a Structure?

- A structure is a **user-defined data type** that groups related variables of different types under a single name.
- Defined using the `struct` keyword.

### Syntax:

```
struct StructureName {
    dataType member1;
    dataType member2;
};
```

### Declaring & Initializing:

```
struct Student s1;                          // declaration
struct Student s1 = {"Riya", 92, 'A'};      // declaration + initialization
```

### Accessing Members:

- Use the **dot operator** (`.`): `s1.name`, `s1.marks`.
- For nested structures: `s1.inner.field` (dot chaining).

### Key Points:

- Structure definitions **must end with a semicolon (`;`)**.
- Strings inside structures should be assigned using `strcpy()`, not directly with `=`.
- An **array of structures** stores multiple records efficiently (e.g., menu items).
- **Nested structures** allow embedding a structure inside another structure for real-world modeling.

## 1 Task 1: Playlist Structure (Song Details)

**Notes:** Declare a structure to represent song details including title, artist, and duration in seconds. Initialize and calculate duration in minutes and remaining seconds.

```
#include <stdio.h>
#include <string.h>

struct Playlist {
    char title[50];
    char artist[50];
```

```
    int duration;   // in seconds
};

int main() {
    // Initialize with favorite song details
    struct Playlist song = {"Tum Hi Ho", "Arijit Singh", 268};

    printf("----- Song Details -----\n");
    printf("Title    : %s\n", song.title);
    printf("Artist   : %s\n", song.artist);
    printf("Duration : %d seconds (%d min %d sec)\n",
           song.duration, song.duration / 60, song.duration % 60);

    return 0;
}
```

**Sample Output:**

```
----- Song Details -----
Title : Tum Hi Ho
Artist : Arijit Singh
Duration : 268 seconds (4 min 28 sec)
```

## 2 Task 2: FoodItem Structure — Array of Structures

**Notes:** Store multiple items in an array of structures to represent a digital menu (Zomato style) and iterate using a loop.

```c
#include <stdio.h>
#include <string.h>

struct FoodItem {
    char itemName[50];
    float price;
    float rating;
};

int main() {
    // Array of 3 FoodItem variables (Zomato-style menu)
    struct FoodItem menu[3] = {
        {"Margherita Pizza", 249.50, 4.5},
        {"Veg Hakka Noodles", 199.00, 4.2},
        {"Chocolate Brownie", 149.75, 4.8}
    };

    printf("------ Zomato Menu ------\n");
    for (int i = 0; i < 3; i++) {
        printf("\nItem %d:\n", i + 1);
        printf("Name   : %s\n", menu[i].itemName);
        printf("Price  : Rs.%.2f\n", menu[i].price);
        printf("Rating : %.1f stars\n", menu[i].rating);
    }

    return 0;
}
```

**Sample Output:**

```
------ Zomato Menu ------

Item 1:
Name : Margherita Pizza
Price : Rs.249.50
Rating : 4.5 stars

Item 2:
Name : Veg Hakka Noodles
Price : Rs.199.00
Rating : 4.2 stars

Item 3:
Name : Chocolate Brownie
Price : Rs.149.75
Rating : 4.8 stars
```

## 3 Task 3: MovieShow — Nested Structure

**Notes:** Combine two structures by embedding `struct Time` inside `struct MovieShow`. Access nested elements using dot chaining (`show.showTime.hours`).

```c
#include <stdio.h>
#include <string.h>

// Nested structure: Time
struct Time {
    int hours;
    int minutes;
};

// Outer structure: MovieShow (contains Time inside)
struct MovieShow {
    char movie[50];
    int screen;
    struct Time showTime;   // nested structure member
};

int main() {
    // Initialize MovieShow variable
    struct MovieShow show = {"Inception", 4, {7, 45}};

    printf("Movie: %s, Screen: %d, Time: %02d:%02d\n",
           show.movie,
           show.screen,
           show.showTime.hours,     // accessing nested member
           show.showTime.minutes);

    return 0;
}
```

**Sample Output:**

```
Movie: Inception, Screen: 4, Time: 07:45
```

## 4 Task 4: InstaProfile — Nested Bio Structure

**Notes:** Model social media profile details where a nested structure holds user bio information (description and age).

```c
#include <stdio.h>
#include <string.h>

// Nested structure: Bio
struct Bio {
    char description[100];
    int age;
};

// Outer structure: InstaProfile
struct InstaProfile {
```

```
    char username[50];
    int followers;
    struct Bio bio;   // nested structure member
};

int main() {
    // Initialize InstaProfile with details
    struct InstaProfile profile = {
        "rohan_codes",
        12500,
        {"Coding | Music | Coffee", 21}
    };

    printf("----- Instagram Profile -----\n");
    printf("Username    : %s\n", profile.username);
    printf("Followers   : %d\n", profile.followers);
    printf("Description : %s\n", profile.bio.description);
    printf("Age         : %d\n", profile.bio.age);

    return 0;
}
```

**Sample Output:**

```
----- Instagram Profile -----
Username : rohan_codes
Followers : 12500
Description : Coding | Music | Coffee
Age : 21
```

# 5 Quick Revision Summary

| Task | Concept | Key Syntax / Pattern |
|---|---|---|
| 1 | Basic Structure | `song.title`, `song.duration` |
| 2 | Array of Structures | `menu[i].price` inside a `for` loop |
| 3 | Nested Structure | `show.showTime.hours` (Dot chaining) |
| 4 | Complex Nested Structure | `profile.bio.description` |

**Golden Rules of Structures in C:**

- Always end structure definitions with a semicolon (`;`).
- Use dot operator (`.`) for direct member access and dot chaining for nested fields.
- For arrays of structures, loop over indices and use `array[i].member`.
- Use `strcpy()` to assign text values to string fields after initialization.